NMK GLOBAL INC.

INTERNAL ENGINEERING DOCUMENTATION

NMK Recruitment Portal

Complete Technical & Operational Documentation

Document Title	NMK Recruitment Portal — Technical Documentation
authors	Mohammed Sohail Ahamed, Gopikiran (AIML team).

Distribution List

Recipient	Role
Saikiran	Job postind use pdf and text
prathyusha	Recruiter anylitics
susmitha	Interview schedule , calendar integration , notifications
sohail	Login,registration and user profile
Gopikiran	Job edit,resume upload,application tracking and frontend layout

Table of Contents

1. **Executive Summary**
2. **Project Scope & Objectives**
3. **Stakeholders & User Roles**
4. **Technology Stack**
5. **System Architecture**
   - 5.1 High-Level Architecture Diagram
   - 5.2 Authentication & Token Flow
   - 5.3 Complete Data Flow Reference (12 flows)
6. **Project & File Structure**
7. **Database Design**
   - 7.1 Entity Relationship Summary
   - 7.2 Model Field Reference
   - 7.3 Enum Types & Status Lifecycles
   - 7.4 Indexing & Performance Notes
8. **Backend API Reference**
   - 8.1 Auth (/auth)
   - 8.2 Jobs (/jobs)
   - 8.3 Applications (/applications)
   - 8.4 Candidate (/candidate)
   - 8.5 Resume (/resume)
   - 8.6 Interviews (/interviews)
   - 8.7 Interviewers (/interviewers)
   - 8.8 Admin & Analytics (/admin)
   - 8.9 Notifications (/notifications)
   - 8.10 Job Shares (/job-shares)
   - 8.11 Job Application Forms (/jobs/{id}/application-form)
   - 8.12 Skills (/skills)
   - 8.13 Job Descriptions (/job-descriptions)
9. **Frontend Pages Reference**
   - 9.1 Candidate Pages
   - 9.2 Recruiter Pages
   - 9.3 Shared Utility Modules
10. **AWS Services Configuration**
    - 10.1 AWS Cognito
    - 10.2 AWS S3
    - 10.3 AWS SES
11. **Security Design**

11.1 Authentication & Token Security
11.2 Authorisation & Role Enforcement
11.3 Data Access Controls
11.4 File Security
11.5 Input Validation

12. Environment Configuration

13. Installation & Local Setup

14. Key User Workflows

15. Complete API Flow Reference (Frontend → Backend → DB)

16. Email Notification System

17. Analytics & Reporting

1 Executive Summary

The NMK Recruitment Portal is a full-stack, cloud-native recruitment management system designed and developed for NMK Global Inc. It provides a complete, end-to-end hiring lifecycle — from job posting and candidate applications through interview scheduling, status tracking, and data-driven analytics — within a single, secure, role-separated platform.

The system eliminates the inefficiencies of fragmented hiring workflows by centralising every hiring touchpoint into one integrated platform. Candidates get a professional portal to discover opportunities, manage their profile, and track application progress in real time. Recruiters get powerful tools to post jobs, build custom application forms, schedule interviews in multiple modes, collaborate with colleagues via job sharing, and monitor the full hiring funnel through live analytics dashboards.

Business Value Delivered

Objective	How This System Achieves It
Centralise recruitment operations	Single platform covers every stage: post → apply → review → interview → offer
Eliminate manual email coordination	AWS SES auto-sends tailored emails at every hiring milestone
Enforce governance & access control	AWS Cognito + role-based route guards ensure strict data separation
Reduce time-to-hire	Live analytics surface bottlenecks; interview scheduling reduces back-and-forth
Provide candidate visibility	Real-time application tracking, notifications, and interview schedules
Enable team collaboration	Recruiters can share jobs; applications assignable to colleagues
Scale with the organisation	Cloud-native on AWS S3, Cognito, SES; PostgreSQL for structured data

System at a Glance

Dimension	Detail
Platform Type	Full-stack web application (SaaS-style, internal)
User Roles	Candidate (job seeker) · Recruiter (NMK hiring staff)
Authentication	AWS Cognito (hosted user pool, email verification, JWT tokens)
Frontend	Python + Streamlit (multi-page, session-based auth)
Backend	Python + FastAPI (RESTful, OpenAPI documented)
Database	PostgreSQL via SQLAlchemy ORM (UUID primary keys)
File Storage	AWS S3 (resumes, profile pictures, job descriptions)
Email	AWS SES (12+ automated email event templates)
Hosting (dev)	localhost:8501 (frontend) · localhost:8000 (backend)
API Documentation	http://localhost:8000/docs (Swagger UI auto-generated)

2 Project Scope & Objectives

In Scope

- Candidate account creation, login, email verification, and password reset via AWS Cognito
- Recruiter account creation restricted to @nmkglobalinc.com domain with company profile
- Candidate profile management: personal info, headline, skills, education, experience, projects, profile picture
- Public shareable candidate profiles accessible without authentication
- Resume upload (PDF/DOC/DOCX, max 5 MB, 10 per account), rename, tag, set primary, preview, delete
- Job posting, editing, archiving, and unarchiving by recruiters
- Job search with keyword, location, and experience filters for candidates
- Custom per-job application forms (text, textarea, select, boolean question types)
- Application submission with custom form answers and resume attachment
- Application status lifecycle: applied → shortlisted → interview → offered / rejected
- Interview scheduling in two modes: Direct (fixed time) and Slot (candidate selects from options)
- Three interview types: Online (meeting link), Offline (location), Telephone
- Calendar (.ics) generation and email delivery for interview events
- Recruiter analytics dashboard with Plotly pipeline charts
- Candidate analytics dashboard with profile score, application breakdown, upcoming interviews
- In-app notification system + AWS SES email for all hiring milestones
- Job sharing between recruiters with email notification to recipients
- Application assignment from one recruiter to another
- Interviewer management (create, list, delete)
- PDF text extraction for uploaded job description files

3 Stakeholders & User Roles

Primary Users

Role	Who They Are	Primary Goals	System Access Level
Candidate	Job seekers applying to NMK-listed roles	Find jobs, apply, track status, manage profile and resumes	Own data only (profile, applications, resumes)
Recruiter	NMK Global hiring staff (@nmkglobalinc.com)	Post jobs, review applicants, schedule interviews, track pipeline	Own jobs + shared jobs + all applicants to their jobs

Role Permissions Matrix

Action	Candidate	Recruiter
View job listings	✅ Yes	✅ Yes
Apply for jobs	✅ Yes	❌ No
Manage own profile	✅ Yes	❌ No
Upload & manage resumes	✅ Yes	❌ No
Track own applications	✅ Yes	❌ No
View candidate profile analytics	✅ Yes	❌ No
Post / edit / archive jobs	❌ No	✅ Yes (own jobs only)
Build custom application forms	❌ No	✅ Yes (own jobs only)
View all applicants for a job	❌ No	✅ Yes (own + shared jobs)
Update application status	❌ No	✅ Yes
Schedule interviews	❌ No	✅ Yes
Assign applications to colleagues	❌ No	✅ Yes (job owner / assigned)
Share jobs with other recruiters	❌ No	✅ Yes (job owner only)
View recruiter analytics dashboard	❌ No	✅ Yes
Confirm interview slot	✅ Yes	❌ No
Manage interviewers list	❌ No	✅ Yes

4 Technology Stack

Layer	Technology	Version	Role in System
Frontend	Python · Streamlit	1.32+	Multi-page web UI, session management, forms
Backend API	Python · FastAPI	0.110+	RESTful API, dependency injection, OpenAPI docs
ORM	SQLAlchemy	2.0+	Database models, relationships, session management
DB Migrations	SQLAlchemy metadata	Auto	Tables auto-created on startup (create_all)
Database	PostgreSQL	14+	Relational storage, UUID primary keys
Authentication	AWS Cognito	Hosted	User pool, JWT issuance, email verification
Token Decode	python-jose (RS256)	3.3+	Verify Cognito JWTs using public JWKS endpoint
File Storage	AWS S3	Boto3	Resumes, profile pictures, job description files
Email	AWS SES	Boto3	Transactional email for all hiring events
Data Validation	Pydantic v2	2.0+	Request / response schema validation
PDF Extraction	pypdf	3.0+	Text extraction from uploaded JD PDF files
Charts	Plotly Express	5.0+	Interactive bar charts in analytics dashboards
HTTP Client	requests	2.31+	Frontend → backend API calls; JWKS fetch
Config	python-dotenv	1.0+	Load .env variables in both frontend and backend
Auth Cookies	extra-streamlit-components	0.18+	Cookie-based token persistence (frontend)

5 System Architecture

5 High-Level Architecture Diagram

The platform follows a clean three-tier architecture. All communication between tiers is authenticated and stateless.

```
┌──────────────────────────────────────────────────────────────────┐
│                      PRESENTATION LAYER                          │
│              Streamlit Frontend  (localhost:8501)                 │
│                                                                   │
│  ┌──────────┐ ┌────────┐ ┌─────────┐ ┌──────────────────┐        │
│  │  login   │ │ signup │ │ profile │ │ jobs / analytics │        │
│  └──────────┘ └────────┘ └─────────┘ └──────────────────┘        │
│            auth.py (session)   layout.py (sidebar)                │
└──────────────────────────┬──────────────────────────────────────┘
                           │  HTTPS  (Authorization: Bearer JWT)
┌──────────────────────────▼──────────────────────────────────────┐
│                      BUSINESS LOGIC LAYER                        │
│               FastAPI Backend  (localhost:8000)                   │
│                                                                   │
│  /auth  /jobs  /candidate  /applications  /interviews             │
│  /resume  /admin  /notifications  /job-shares  /skills            │
│  /interviewers  /job-descriptions  /job-application-forms         │
│                                                                   │
│  decode_cognito_token()  →  get_current_user()                    │
│  get_current_candidate() | get_current_recruiter()                │
└──────────────────────────────────────────────────────────────────┘
        │                    │
┌───────▼──────────┐   ┌──────▼──────────────────────────────────┐
│ PostgreSQL       │   │              AWS Cloud Services          │
│                  │   │                                          │
│  Users           │   │  Cognito  →  User pool, JWT, email verify │
│  Jobs            │   │  S3       →  Resumes, pictures, JD files  │
│  Candidates      │   │  SES      →  All transactional emails     │
│  Applications    │   │                                          │
│  Interviews      │   └──────────────────────────────────────────┘
│  Resumes         │
│  Notifs          │
└──────────────────┘
```

6 Project & File Structure

```
recruitment_db/
├── Backend/
│   └── app/
│       ├── __init__.py                  ← Package init
│       ├── main.py                      ← FastAPI app, router registration, DB init
│       ├── db.py                        ← SQLAlchemy engine, SessionLocal, get_db()
│       ├── models.py                    ← All ORM table definitions
│       ├── schemas.py                   ← All Pydantic request/response schemas
│       │
│       ├── auth_api.py                  ← POST /auth/complete-login, JWT decode
│       ├── jobs_api.py                  ← CRUD for jobs, search, applicant views
│       ├── candidate_api.py             ← Profile, skills, education, exp, projects
│       ├── application_api.py           ← Apply, my-apps, status update
│       ├── interview_api.py             ← Schedule, reschedule, cancel, slots
│       ├── interviewer_api.py           ← Interviewer CRUD
│       ├── resume_api.py                ← Upload, list, rename, primary, delete
│       ├── admin_api.py                 ← Analytics, pipeline, assignment
│       ├── notification_api.py          ← Get notifications, mark read
│       ├── notification_utils.py        ← create_notification() utility
│       ├── job_application_forms_api.py ← Save/get custom form questions
│       ├── job_descriptions.py          ← JD text, file upload (PDF → S3)
│       ├── job_share_api.py             ← Share jobs, list shared-with-me
│       ├── skills_api.py                ← Skill master list search
│       ├── email_utils.py               ← SES client, send_email, attachments
│       ├── email_templates.py           ← All (subject, body) template functions
│       ├── calendar_utils.py            ← generate_interview_ics() .ics builder
│       ├── pdf_text.py                  ← pypdf text extraction helper
│       └── utils/
│           └── s3.py                   ← Upload, delete, presign URL helpers
│
├── Frontend/
│   ├── app.py                           ← Landing page (root entry point)
│   ├── auth.py                          ← Session helpers: require_login/role
│   ├── layout.py                        ← render_sidebar() navigation
│   ├── assets/
│   │   └── nmk_logo.png                ← Company logo
│   └── pages/
│       ├── login.py                     ← Login + forgot password
│       ├── 1_signup.py                  ← Registration (both roles)
│       ├── 3_candidate_profile.py       ← Full profile editor
│       ├── 4_job_listings.py            ← Job search & apply
│       ├── 5_resume_upload.py           ← Resume management
│       ├── 6_my_applications.py         ← Application tracker
│       ├── 7_profile_analytics.py       ← Candidate analytics
│       ├── job_application_form.py      ← Dynamic application form
│       ├── recruiter_dashboard.py       ← Job posting & management
│       ├── recruiter_application_form.py ← Form builder
│       ├── recruiter_analytics.py       ← Recruiter analytics
│       ├── recruiter_candidate_profile.py← View candidate (recruiter side)
│       ├── shared_jobs.py               ← Jobs shared with recruiter
```

```
│       ├── notifications.py          ← Notification centre
│       └── public_profile.py         ← Public candidate profile page
│
├── .env                              ← Environment variables (NOT in git)
└── requirements.txt                  ← Top-level dependencies
```

7 Database Design

7.1 Entity Relationship Summary

Model / Table	Primary Key	Relationships	Key Purpose
users	UUID	Has one candidate_profiles OR recruiters	Root identity record for all portal users
candidate_profiles	UUID	Belongs to users; has many skills, edu, exp, resumes	Full candidate professional profile
recruiters	UUID	Belongs to users and companies	Recruiter identity linked to a company
companies	UUID	Has many recruiters	Organisation that recruiters belong to
jobs	UUID	Belongs to recruiters; has many applications, shares	Job postings created by recruiters
applications	UUID	Links candidate_profiles ↔ jobs; has interviews	A candidate's submission for a job
job_application_questions	UUID	Belongs to jobs	Custom form questions per job
job_application_answers	UUID	Belongs to applications + questions	Candidate answers to custom questions
interviews	UUID	Belongs to applications; has slots + interviewers	Interview event for an application
interview_slots	UUID	Belongs to interviews	Individual time options in slot mode
interviewers	UUID	Many-to-many with interviews	People conducting the interview
resumes	UUID	Belongs to candidate_profiles	Uploaded resume files (S3 key + metadata)
skills	UUID	Master list; many-to-many via candidate_skills/job_skills	Skill taxonomy
candidate_skills	UUID	Links candidate_profiles ↔ skills	Candidate's skill with proficiency
job_skills	UUID	Links jobs ↔ skills	Skills required for a job
job_descriptions	UUID	Standalone; linked optionally to jobs	Structured JD with text + S3 file

Model / Table	Primary Key	Relationships	Key Purpose
job_shares	UUID	Links jobs to multiple recruiters	Grants shared access to a job
notifications	UUID	Belongs to users	In-app notification messages

7.2 Key Model Field Reference

users

Field	Type	Notes
id	UUID (PK)	Auto-generated
cognito_sub	String	AWS Cognito user sub — used for JWT lookup
email	String	Unique. Indexed.
full_name	String	Synced from Cognito custom:full_name attribute
role	UserRole	Enum: 'user' | 'recruiter'
is_active	Boolean	Default True. Deactivated accounts cannot log in
created_at	DateTime	Auto-set on INSERT

applications

Field	Type	Notes
id	UUID (PK)	Auto-generated
job_id	UUID (FK → jobs)	The job being applied to
candidate_id	UUID (FK → candidate_profiles)	Applicant
resume_id	UUID (FK → resumes)	Resume used for this application
status	ApplicationStatus	Enum. Default: 'applied'
applied_at	DateTime	Auto-set on INSERT
assigned_recruiter_id	UUID (FK → recruiters)	Nullable. Set when assigned

Field	Type	Notes
assigned_by	UUID (FK → users)	Who performed the assignment
assigned_at	DateTime	When assignment was made
candidate_notes	Text	Optional internal notes

interviews

Field	Type	Notes
id	UUID (PK)	Auto-generated
application_id	UUID (FK → applications)	The application this interview belongs to
scheduled_at	DateTime	NULL in slot mode until candidate confirms
interview_type	String	'online' | 'offline' | 'telephone'
schedule_mode	String	'direct' | 'slots'
meeting_link	String	For online interviews
location	String	For offline interviews
status	InterviewStatus	Enum: scheduled | completed | cancelled | rescheduled
created_at	DateTime	Auto-set on INSERT

7.3 Enum Types & Status Lifecycles

Enum	Values
UserRole	user | recruiter
JobStatus	draft | active | closed | archived
ApplicationStatus	applied → shortlisted → interview → offered | rejected
InterviewStatus	scheduled | completed | cancelled | rescheduled

```
APPLICATION STATUS LIFECYCLE:

applied ──────► shortlisted ──────► interview ──────► offered
   │                  │                  │
   └──────────────────┴──────────────────► rejected  (from any stage)
```

7.4 Indexing & Performance Notes

- cognito_sub on users is the critical lookup field — every API call queries by it. Ensure this column is indexed.
- email on users is unique and indexed for O(1) duplicate detection on signup.
- candidate_id on applications is frequently queried (my-applications endpoint). Index recommended.
- job_id on applications is heavily queried for recruiter views. Index recommended.
- status on applications is used in GROUP BY queries for analytics. Partial index on common values may help at scale.
- Tables are auto-created by SQLAlchemy's Base.metadata.create_all(). For production, migrate to Alembic for controlled schema migrations.

8 Backend API Reference

Base URL: http://localhost:8000 | Interactive Docs: http://localhost:8000/docs | All protected routes require: Authorization: Bearer {IdToken}

> **Token Type**
> The IdToken (not AccessToken) from Cognito is used as the Bearer token. It contains the custom claims (role, full_name, etc.) needed by the backend.

8.1 Auth — /auth

Method	Endpoint	Auth	Request Body / Params	Returns	Notes
POST	/auth/complete-login	Bearer	Header only	{ user_id, role, recruiter_id }	Call after Cognito login. Upserts User, creates profile/recruiter.
POST	/auth/forgot-password	None	{ email }	{ message }	Triggers Cognito password reset email.
POST	/auth/confirm-reset-password	None	email, confirmation_code, new_password	{ message }	Confirms new password with reset code.

8.2 Jobs — /jobs

Method	Endpoint	Auth	Request Body / Params	Returns	Notes
GET	/jobs/search	Bearer	?keyword, ?location, ?min_experience	[JobRead]	Candidate job search with filters
GET	/jobs/my	Bearer	None	[JobRead]	Recruiter: all jobs they posted
POST	/jobs	Bearer	{ title, description, location, experience, salary, skills[] }	JobRead	Create new job posting
PUT	/jobs/{job_id}	Bearer	JobUpdate (partial fields + skills[])	JobRead	Update job. Replaces skills.
GET	/jobs/{id}/applicants	Bearer	Path: job_id	[ApplicationRead]	All applicants for a job

8.3 Applications — /applications

Method	Endpoint	Auth	Request Body / Params	Returns	Notes
POST	/applications/apply	Bearer	multipart: job_id, answers (JSON str), resume_id OR resume_file	{ message, app_id }	Submit application. Creates DB rows, uploads file, sends email.
GET	/applications/my	Bearer	None	[ApplicationRead]	Candidate: all their applications with job and status
PUT	/applications/{id}/status	Bearer	{ status }	{ message, new_status }	Update status. Triggers SES email + notification.
GET	/applications/{id}/answers	Bearer	Path: application_id	[{question, answer}]	View candidate's form answers for this application

8.4 Candidate — /candidate

Method	Endpoint	Auth	Request Body / Params	Returns	Notes
GET	/candidate/profile	Bearer	None	CandidateProfileRead	Full profile for current user
PUT	/candidate/profile	Bearer	CandidateProfileUpdate	CandidateProfileRead	Update profile fields
POST	/candidate/profile-picture	Bearer	multipart: file (image)	{ message, url }	Upload picture to S3. Saves key to profile.
GET	/candidate/profile-analytics	Bearer	None	ProfileAnalytics	Views, apps, breakdown, score, interviews
GET	/candidate/public/{username}	None	Path: public_username	Public profile JSON	No auth. Only if visibility='public'.
GET	/candidate/candidate/{id}	Bearer	?application_id	Full candidate data	Recruiter view. Scoped to application.
POST	/candidate/skills	Bearer	{ name, proficiency, years }	Skill	Add skill to profile
DELETE	/candidate/skills/{skill_id}	Bearer	Path: skill_id	{ message }	Remove skill

Method	Endpoint	Auth	Request Body / Params	Returns	Notes
POST	/candidate/education	Bearer	{ institution, degree, years }	Education	Add education entry
DELETE	/candidate/education/{id}	Bearer	Path: edu_id	{ message }	Remove education entry
POST	/candidate/experience	Bearer	{ company, role, dates, is_current }	Experience	Add experience entry
DELETE	/candidate/experience/{id}	Bearer	Path: exp_id	{ message }	Remove experience entry
POST	/candidate/projects	Bearer	{ title, description, url }	Project	Add project
GET	/candidate/resume/access/application/{id}	Bearer	Path: application_id	{ url }	Presigned resume URL (recruiter access)
GET	/candidate/media/profile-picture/{key}	Bearer	Path: s3_key + ?application_id	{ url }	Presigned profile picture URL

8.5 Resume — /resume

Method	Endpoint	Auth	Request Body / Params	Returns	Notes
POST	/resume/upload	Bearer	multipart: file	{ message, resume_id }	Upload PDF/DOC/DOCX. Max 5MB, 10 per user.
GET	/resume/my-resumes	Bearer	None	{ resumes: [] }	List with metadata, tags, share count
GET	/resume/access/{id}	Bearer	Path: resume_id	{ url }	Presigned S3 URL. Increments share_count.
POST	/resume/set-primary/{id}	Bearer	Path: resume_id	{ message }	Clears all primaries, sets this one
PATCH	/resume/rename/{id}	Bearer	?name	{ message }	Update display_name
PATCH	/resume/tags/{id}	Bearer	?tags	{ message }	Update tags string
DELETE	/resume/delete/{id}	Bearer	Path: resume_id	{ message }	Deletes from S3 and DB

8.6 Interviews — /interviews

Method	Endpoint	Auth	Request Body / Params	Returns	Notes
POST	/interviews/schedule	Bearer	ScheduleInterviewRequest (see schemas)	{ interview_id, scheduled_at }	Direct or slot mode. Sends .ics + email.
GET	/interviews/my	Bearer	None	[InterviewRead]	All upcoming interviews for current user
PUT	/interviews/{id}/reschedule	Bearer	{ scheduled_at, meeting_link?, location? }	{ message }	Update time. Emails candidate.
PUT	/interviews/{id}/cancel	Bearer	Path: interview_id	{ message }	Cancel. Emails candidate.
PUT	/interviews/{id}/confirm-slot	Bearer	{ slot_id }	{ message }	Candidate confirms slot. Sends email + .ics.

8.7 Interviewers — /interviewers

Method	Endpoint	Auth	Request Body / Params	Returns	Notes
GET	/interviewers/	Bearer	None	[{id, name, email}]	List all saved interviewers
POST	/interviewers/	Bearer	{ name, email }	{ id, name, email }	Create interviewer. Email must be unique.
DELETE	/interviewers/{id}	None	Path: interviewer_id	{ message }	Delete interviewer

8.8 Admin & Analytics — /admin

Method	Endpoint	Auth	Request Body / Params	Returns	Notes
GET	/admin/recruiter/applications-per-job	Bearer	None	[{job_id, job_title, applications}]	Jobs owned + shared with counts
GET	/admin/application-status-summary	Bearer	None	{ applied, shortlisted, ... }	All status counts aggregated

Method	Endpoint	Auth	Request Body / Params	Returns	Notes
GET	/admin/upcoming-interviews	Bearer	None	[{candidate, job_title, scheduled_at}]	From now() onwards
GET	/admin/candidates-needing-action	Bearer	None	[{name, job, status, days_pending}]	Top 10 longest pending
GET	/admin/job-performance	Bearer	None	[{job_id, job_title, applications}]	Per-job app volume
PUT	/admin/assign-application/{id}	Bearer	?recruiter_id	{ message }	Reassign application to colleague
GET	/admin/recruiters	Bearer	None	[{id, full_name}]	Recruiters in same company

8.9 Notifications — /notifications

Method	Endpoint	Auth	Request Body / Params	Returns	Notes
GET	/notifications/	Bearer	None	[Notification]	All for current user, newest first
PUT	/notifications/{id}/read	Bearer	Path: notification_id	{ message }	Mark as read

8.10 Job Shares — /job-shares

Method	Endpoint	Auth	Request Body / Params	Returns	Notes
POST	/job-shares/{job_id}/share	Bearer	Body: [recruiter_id1, recruiter_id2]	{ message }	Share job. Sends email to each recipient.
GET	/job-shares/shared-with-me	Bearer	None	[{job_id, title, location}]	Jobs shared with current recruiter

8.11 Job Application Forms — /jobs

Method	Endpoint	Auth	Request Body / Params	Returns	Notes
POST	/jobs/{job_id}/application-form	Bearer	[JobApplicationQuestionCreate]	[saved questions]	Replaces existing form. Recruiter only.
GET	/jobs/{job_id}/application-form	None	Path: job_id	[JobApplicationQuestionRead]	Returns ordered questions for candidates

8.12 Skills — /skills

Method	Endpoint	Auth	Request Body / Params	Returns	Notes
GET	/skills	None	?search={text}	[{id, name}]	Returns up to 10 matching skills from master list

8.13 Job Descriptions — /job-descriptions

Method	Endpoint	Auth	Request Body / Params	Returns	Notes
POST	/job-descriptions	Bearer	JobDescriptionCreate	JobDescriptionRead	Create structured JD record
POST	/job-descriptions/upload	Bearer	multipart: file (PDF)	{ file_key, extracted_text }	Upload JD PDF → extract text → save to job
GET	/job-descriptions	None	None	[JobDescriptionRead]	List all active JDs
PUT	/job-descriptions/{jd_id}	Bearer	JobDescriptionCreate	JobDescriptionRead	Update JD content
GET	/job-descriptions/file/{key}	Bearer	Path: file_key	{ url }	Presigned S3 URL for JD file

9 Frontend Pages Reference

9.1 Candidate Pages

Page / File	Route / Nav Label	Auth Required	Key Functionality
app.py	/ (root)	No	Public landing page. Hero section, feature overview, Get Started / Login CTAs.
login.py	/pages/login	No	Cognito login form, forgot password flow with reset code, redirect by role on success.
1_signup.py	/pages/1_signup	No	Registration for both roles. Recruiter fields appear conditionally. Email verification step.
3_candidate_profile.py	Profile	Candidate	Full profile editor: personal info, headline, skills, education, experience, projects, photo, public username, visibility toggle.
4_job_listings.py	Jobs Search	Candidate	Job search with keyword/location/experience filters. Apply button → form. Disabled button for already-applied jobs.
job_application_form.py	(internal page)	Candidate	Dynamic form: renders recruiter-defined questions. Resume picker from uploaded list or new upload.
5_resume_upload.py	Resumes	Candidate	Upload, list, rename, tag, set primary, preview (presigned URL), share link, delete resumes.
6_my_applications.py	My Applications	Candidate	All submitted applications with status badges, applied date, job details.
7_profile_analytics.py	Profile Analytics	Candidate	Profile score, views, application breakdown progress bars, interview schedule, insight percentages.
notifications.py	Notifications	Candidate	All notifications ordered newest first with title, message, and timestamp.
public_profile.py	?username=...	No	Public-facing read-only profile. Header, about, skills, experience, education, projects.

9.2 Recruiter Pages

Page / File	Route / Nav Label	Auth Required	Key Functionality
recruiter_dashboard.py	Job Posting	Recruiter	Post/edit/archive jobs, view applicants per job, update status, view candidate profiles, share job with colleagues.
recruiter_application_form.py	Application Form	Recruiter	Build custom forms per job: add text/textarea/select/boolean questions, set order and required flag, save form.
recruiter_analytics.py	Recruiter Analytics	Recruiter	Pipeline bar chart (Plotly), metrics, per-job counts, upcoming interviews table, candidates needing action table.
recruiter_candidate_profile.py	(internal page)	Recruiter	Read-only candidate view: contact info, skills, education, experience, projects, resume preview via presigned URL.
shared_jobs.py	Shared With Me	Recruiter	Jobs shared by other recruiters. Each item links to that job's applicant list.

9.3 Shared Utility Modules

Module	Functions / Role
auth.py	require_login() — redirects to app.py if no id_token in session. require_role(role) — stops page if wrong role. auth_headers() — returns {Authorization: Bearer token} for API calls. logout() — clears session and redirects to app.py.
layout.py	render_sidebar() — draws role-conditional sidebar: logo, section headings, navigation buttons (st.switch_page), and Logout. Called on every protected page first.

10 AWS Services Configuration

10.1 AWS Cognito

Setting	Value / Detail
Service	AWS Cognito User Pool
Auth Flow	USER_PASSWORD_AUTH (must be enabled on App Client)
Token Types Used	IdToken for backend authentication (contains custom claims)
Custom User Attributes	custom:role, custom:full_name, custom:company_name, custom:industry, custom:website, custom:location, custom:designation
Email Verification	Required before first login. Cognito sends 6-digit code.
Password Reset	Cognito-hosted forgot_password() + confirm_forgot_password() flow
Domain Restriction	Recruiter emails must end in @nmkglobalinc.com — enforced in frontend
JWKS Endpoint	https://cognito-idp.{region}.amazonaws.com/{pool_id}/.well-known/jwks.json
Token Signing	RS256 — asymmetric signature; public keys fetched and cached

10.2 AWS S3

S3 Prefix / Folder	Contents	Max Size	Access Method
resumes/{candidate_id}/	Candidate resume files (PDF, DOC, DOCX)	5 MB per file	Presigned GET URL (1hr TTL)
profile-pictures/	Candidate profile photos (JPEG, PNG)	5 MB	Presigned GET URL (1hr TTL)
job_descriptions/	Uploaded JD files + extracted text	No hard limit	Presigned GET URL (1hr TTL)

10.3 AWS SES — Email Events

Trigger Event	Template Function	Recipient	Includes Attachment
Application submitted	application_confirmation()	Candidate	No
Shortlisted	shortlisted()	Candidate	No
Rejected	rejected()	Candidate	No
Offer extended	offer()	Candidate	No
Interview scheduled (direct)	interview_scheduled()	Candidate	Yes — .ics calendar file
Interview slots sent	interview_slots_shared()	Candidate	No
Slot confirmed by candidate	interview_slot_confirmed()	Candidate	Yes — .ics calendar file
Interview rescheduled	interview_rescheduled()	Candidate	No
Resume uploaded	resume_uploaded()	Candidate	No
Application assigned	recruiter_assigned()	Candidate	No
Job shared with recruiter	job_shared()	Target Recruiter	No

11 Security Design

11.1 Authentication & Token Security

- All protected API routes use FastAPI's OAuth2PasswordBearer scheme — the IdToken is extracted from the Authorization header on every request.
- Tokens are RS256-signed JWTs. The public JWKS keys are fetched from Cognito's well-known endpoint on first use and cached in memory to avoid repeated network calls.
- Expired tokens are detected by python-jose and return HTTP 401 'Token expired. Please login again.' — no stale auth possible.
- Invalid signatures or malformed tokens return HTTP 401 'Invalid authentication token'.
- The Streamlit frontend stores the IdToken in Streamlit session state (server-side memory), not in localStorage or cookies by default, reducing XSS exposure.

11.2 Authorisation & Role Enforcement

- Role enforcement occurs at two levels: (1) frontend — require_role() stops page rendering before any API call is made; (2) backend — route-level dependencies get_current_candidate() and get_current_recruiter() return HTTP 403 for wrong role.
- Dual enforcement means a compromised frontend session cannot bypass backend access controls.

11.3 Data Access Controls

Resource	Access Rule
Candidate profile	Readable only by the owning candidate (authenticated) or public if visibility='public' (via /candidate/public/{username})
Resume files	Owner candidate can access always. Recruiters access only via application_id — scoped to their specific application
Applications	Candidates see only their own. Recruiters see only applications to their owned or shared jobs
Jobs	All active jobs searchable by candidates. Recruiters manage only jobs they posted (or were shared with them)
Interview scheduling	Only the job owner or assigned recruiter can schedule or update interviews
Assignment	Only job owner or currently assigned recruiter can reassign an application
Job sharing	Only the job owner (original poster) can share a job with colleagues

Resource	Access Rule
Analytics	Candidate analytics: own data only. Recruiter analytics: own + shared jobs

11.4 File Security

- File type validation: MIME type checked server-side on upload, not just file extension.
- File size capped at 5 MB per resume to prevent storage abuse.
- Maximum 10 resumes per candidate — enforced at upload endpoint.
- Presigned URLs expire after 1 hour. A shared link cannot be used after expiry.
- S3 bucket must have Block Public Access enabled. Files are never served directly.

11.5 Input Validation

- All request bodies are validated by Pydantic v2 schemas — malformed JSON or missing required fields return HTTP 422 with detailed error messages.
- Email domain validation for recruiter signup happens on the frontend before Cognito API call and again implicitly via the recruiter email format.
- SQL injection is not possible via SQLAlchemy ORM — all queries use parameterised statements.
- Optional: add rate limiting to auth endpoints to prevent brute-force attacks.

12 Environment Configuration

All secrets and environment-specific settings are loaded from a .env file via python-dotenv. Never commit this file to version control.

🔒 **Security Notice**

The .env file contains AWS credentials, database connection strings, and Cognito secrets. Add .env to .gitignore immediately. Use AWS Secrets Manager or equivalent in production.

Complete .env Reference

Variable	Used By	Required
DATABASE_URL	Backend (db.py)	Yes
AWS_REGION	Backend + Frontend	Yes
COGNITO_USER_POOL_ID	Backend (auth_api.py)	Yes
COGNITO_CLIENT_ID	Backend + Frontend	Yes
AWS_ACCESS_KEY_ID	Backend (S3, SES)	Yes
AWS_SECRET_ACCESS_KEY	Backend (S3, SES)	Yes
S3_BUCKET_NAME	Backend (S3 uploads)	Yes
SES_SENDER_EMAIL	Backend (email_utils.py)	Yes
API_BASE_URL	Frontend (all API calls)	Yes
AWS_S3_PROFILE_PICTURES_PREFIX	Backend (s3.py)	Optional
AWS_S3_RESUMES_PREFIX	Backend (s3.py)	Optional

13 Installation & Local Setup

Prerequisites

Prerequisite	Version	Purpose
Python	3.11+	Backend and frontend runtime
PostgreSQL	14+	Database server — must be running before backend starts
AWS Account	Active	Cognito, S3, SES must be configured
pip	Latest	Python package manager
Virtual Environment	venv	Isolate dependencies per service

Step 1 — Clone & Configure

```
# Clone the repository
git clone <repository-url>
cd recruitment_db

# Create .env in the project root (see Section 12 for all variables)
cp .env.example .env
# Edit .env with your actual values
```

Step 2 — Backend Setup

```
cd Backend

# Create and activate virtual environment
python -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate

# Install all Python dependencies
pip install -r requirements.txt

# Verify PostgreSQL is running and DATABASE_URL is correct
# Tables are auto-created by SQLAlchemy on first startup

# Start the FastAPI backend server
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000

# ✓ API available at:       http://localhost:8000
# ✓ Swagger UI available at: http://localhost:8000/docs
# ✓ ReDoc available at:     http://localhost:8000/redoc
```

Step 3 — Frontend Setup

```
cd ../Frontend

# Create and activate virtual environment
python -m venv venv
source venv/bin/activate

# Install Streamlit and all frontend dependencies
pip install -r requirements.txt

# Start the Streamlit frontend
streamlit run app.py

# ✓ App available at: http://localhost:8501
```

Step 4 — Verify Setup

1. Open http://localhost:8000/docs — confirm all routers load without error
2. Open http://localhost:8501 — confirm the NMK landing page loads
3. Register a test candidate account and verify the email confirmation arrives
4. Log in — confirm redirect to candidate profile page
5. Register a test recruiter account with an @nmkglobalinc.com email
6. Log in as recruiter — confirm redirect to recruiter dashboard

> ✅ **Quick Verification**
> Run GET http://localhost:8000/health (if implemented) or GET http://localhost:8000/docs. A 200 response confirms the backend is running and connected to PostgreSQL.

14 Key User Workflows

14.1 Candidate: First-Time Onboarding

Step	Action	System Response
1	Visit landing page at http://localhost:8501	Landing page loads. Login and Get Started buttons visible.
2	Click Get Started → signup page	Registration form shown with role selector.
3	Select role = 'user', fill name, email, password, click Create Account	Cognito account created (unconfirmed). Verification code emailed.
4	Enter verification code from email	Account confirmed in Cognito. Success message.
5	Login with email and password	Cognito returns tokens. Backend syncs user and creates CandidateProfile. Redirect to Profile page.
6	Complete profile (headline, summary, skills, education, experience)	Profile updated in DB. Profile score increases with each section.
7	Upload resume via Resumes page	File uploaded to S3. Resume row created. Confirmation email sent.
8	Search jobs, click Apply, fill form, submit	Application created. Answers saved. Email sent. Job marked applied.

14.2 Recruiter: Full Hiring Cycle

Step	Action	System Response
1	Login with @nmkglobalinc.com email	Tokens issued. Backend creates Recruiter + Company on first login. Redirect to dashboard.
2	Click 'Post Job', fill title/location/salary/skills, submit	Job created with status='active'. Appears in candidate search immediately.
3	Go to Application Form Builder, select job, add questions, save	Questions saved. Candidates see these questions when applying for this job.
4	Candidate applies for the job	Application appears in recruiter's applicant list.
5	View applicant list, click candidate name	Candidate profile opens. Profile view recorded.
6	View candidate's resume via Preview Resume button	Presigned S3 URL generated. Resume opens in browser.

Step	Action	System Response
7	Update status to 'shortlisted'	Status updated. SES 'Shortlisted' email sent to candidate. In-app notification created.
8	Schedule interview (direct mode, online)	Interview created. .ics file generated. Email with calendar invite sent to candidate.
9	Update status to 'offered'	Status updated. SES offer letter email sent.
10	Check analytics page	Pipeline chart and metrics update to reflect the new hire.

14.3 Interview Slot Selection (Candidate)

Step	Action	System Response
1	Recruiter schedules with schedule_mode='slots' and provides multiple times	InterviewSlot rows created. Candidate emailed 'Select Interview Slot' with portal link.
2	Candidate logs in, goes to My Applications	Application shows slot picker UI with available times.
3	Candidate selects preferred slot and confirms	Slot marked confirmed. Interview.scheduled_at set. SES 'Slot Confirmed' + .ics sent to candidate.
4	Recruiter sees confirmed slot in their upcoming interviews	Interview listed in analytics dashboard upcoming interviews table.

15 Complete API Flow Reference (Frontend → Backend → DB)

Every frontend user action is mapped below: what is sent, which endpoint handles it, what the backend does to the database and AWS services, and what is returned to the UI.

15.1 Authentication

Frontend Action	Data Sent to Backend	API Endpoint	Response / Side Effects
Signup form submitted (candidate)	Cognito SDK: {email, password, custom:role='user', custom:full_name, phone}	Cognito cognito.sign_up()	Unconfirmed account created. Verification code emailed. show_verify=True in session.
Verification code entered	Cognito SDK: {email, confirmationCode}	Cognito confirm_sign_up()	Account confirmed. verify_success=True shown. User can log in.
Login form submitted	Cognito SDK: {USERNAME, PASSWORD}	Cognito initiate_auth()	Returns IdToken + AccessToken. Both stored in session_state.
Post-login backend sync	Header: Bearer {IdToken}	POST /auth/complete-login	Decodes JWT. Upserts User. Creates CandidateProfile or Recruiter+Company. Returns {user_id, role, recruiter_id}.
Forgot password — send code	Cognito SDK: {email}	Cognito forgot_password()	Cognito emails 6-digit reset code.
Forgot password — confirm new	Cognito SDK: {email, code, newPassword}	Cognito confirm_forgot_passw ord()	Password updated. User shown success. Redirected to login.

15.2 Candidate Profile

Frontend Action	Data Sent to Backend	API Endpoint	Response / Side Effects
Profile page loads	Bearer token	GET /candidate/profile	Returns full profile object: personal info, skills[], education[], experience[], projects[].
Save profile changes	Bearer + JSON: {summary, location, experience, links,	PUT /candidate/profile	DB row updated. Returns updated profile. Profile score recalculates.

Frontend Action	Data Sent to Backend	API Endpoint	Response / Side Effects
	visibility, public_username, ...}		
Upload profile picture	Bearer + multipart: image file	POST /candidate/profile-picture	Uploaded to S3 profile-pictures/{uuid}.ext. Key saved to profile. Returns {url}.
Search skills autocomplete	Bearer + ?search={text}	GET /skills?search={text}	Returns up to 10 matching Skill records [{id, name}].
Add skill	Bearer + JSON: {name, proficiency, years}	POST /candidate/skills	Creates CandidateSkill row. Returns saved skill.
Delete skill	Bearer + path: skill_id	DELETE /candidate/skills/{id}	Deletes CandidateSkill row. Returns {message}.
Add education	Bearer + JSON: {institution, degree, field, years}	POST /candidate/education	Creates CandidateEducation row. Returns saved object.
Add experience	Bearer + JSON: {company, role, dates, is_current}	POST /candidate/experience	Creates CandidateExperience row. Returns saved object.
Add project	Bearer + JSON: {title, description, url}	POST /candidate/projects	Creates CandidateProject row. Returns saved object.
View public profile	No auth + ?username={username}	GET /candidate/public/{username}	Returns public-safe profile. Only if visibility='public'. No auth required.

15.3 Resume Management

Frontend Action	Data Sent to Backend	API Endpoint	Response / Side Effects
Resume page loads	Bearer token	GET /resume/my-resumes	Returns {resumes: [{resume_id, filename, display_name, tags, uploaded_at, is_primary, share_count}]}.
Upload resume	Bearer + multipart: PDF/DOC/DOCX ≤5MB	POST /resume/upload	Validates type + size. Uploads to S3. Creates Resume row. Sends SES email. Returns {resume_id}.
Preview or Share button clicked	Bearer + path: resume_id	GET /resume/access/{id}	Increments share_count. Updates last_accessed_at. Generates presigned S3 URL (1hr). Returns {url}.

Frontend Action	Data Sent to Backend	API Endpoint	Response / Side Effects
Set as Primary	Bearer + path: resume_id	POST /resume/set-primary/{id}	Sets all candidate's resumes is_primary=False, then this one True. Returns {message}.
Save renamed name	Bearer + path: resume_id + ?name={text}	PATCH /resume/rename/{id}	Updates display_name field. Returns {message}.
Confirm delete	Bearer + path: resume_id	DELETE /resume/delete/{id}	Deletes from S3 (s3.delete_object). Deletes Resume DB row. Returns {message}.

15.4 Job Search & Application

Frontend Action	Data Sent to Backend	API Endpoint	Response / Side Effects
Job listings page loads	Bearer + ?keyword, ?location, ?min_experience	GET /jobs/search	Returns matching active jobs [{job_id, title, company_name, location, salary, experience, description, skills[]}].
Applied-jobs sync on load	Bearer token	GET /applications/my	Returns all applications. Frontend extracts job IDs into applied_jobs set. Applied jobs show disabled button.
Application form page loads	Bearer + path: job_id	GET /jobs/{id}/application-form	Returns ordered question list [{id, question_text, field_type, options, is_required, order_index}].
Form page loads resumes	Bearer token	GET /resume/my-resumes	Returns resume list for selector dropdown.
Submit application	Bearer + multipart: job_id, answers (JSON str), resume_id or resume_file	POST /applications/apply	Creates Application row. Saves Answers. Uploads file (if new). Sends SES email. Creates Notification. Returns {app_id}.
My Applications page loads	Bearer token	GET /applications/my	Returns [{id, job: {title, company}, status, applied_at, resume_id}].

15.5 Profile Analytics (Candidate)

Frontend Action	Data Sent to Backend	API Endpoint	Response / Side Effects
Analytics page loads	Bearer token (cached 120s)	GET /candidate/profile-analytics	Returns {profile_views, total_applications, application_breakdown {applied/shortlisted/interview/offered/rejected}, profile_score (0-100), upcoming_interviews [{job_title, date, time}]}.

15.6 Recruiter: Job Management

Frontend Action	Data Sent to Backend	API Endpoint	Response / Side Effects
Recruiter dashboard loads	Bearer token	GET /jobs/my	Returns all jobs posted by this recruiter with status, skills, dates.
Post new job	Bearer + JSON: {title, description, location, experience, salary, skills[]}	POST /jobs	Creates Job row + JobSkill rows. Status='active'. Returns JobRead.
Upload JD PDF	Bearer + multipart: PDF file	POST /job-descriptions/upload	Extracts text via pypdf. Uploads to S3. Updates job.description with extracted text. Returns {file_key, extracted_text}.
Edit job	Bearer + JSON: partial JobUpdate + skills[]	PUT /jobs/{job_id}	Updates Job row. Replaces JobSkill rows. Returns updated JobRead.
Archive job	Bearer + JSON: {status: 'archived'}	PUT /jobs/{job_id}	Sets status='archived', is_active=False. Job disappears from candidate search.
View applicants	Bearer + path: job_id	GET /jobs/{job_id}/applicants	Returns all application records with candidate summary and current status.
Update application status	Bearer + JSON: {status}	PUT /applications/{id}/status	Updates DB status. Triggers SES email (shortlisted/rejected/offered/interview). Creates Notification.

15.7 Recruiter: Candidate Profile View

Frontend Action	Data Sent to Backend	API Endpoint	Response / Side Effects
Click candidate name	Bearer + path: candidate_id + ?application_id	GET /candidate/candidate/{id}	Returns {candidate: {name, email, phone, experience}, skills[], education[], experience[], projects[], resume: {filename, id}}.
Preview Resume button	Bearer + path: application_id	GET /candidate/resume/access/application/{id}	Validates recruiter has access via application. Generates presigned S3 URL for resume. Returns {url}.
View profile picture	Bearer + path: s3_key + ?application_id	GET /candidate/media/profile-picture/{key}	Validates access. Generates presigned S3 URL for profile image. Returns {url}.

15.8 Interview Scheduling

Frontend Action	Data Sent to Backend	API Endpoint	Response / Side Effects
Schedule (direct mode)	Bearer + JSON: {application_id, schedule_mode:'direct', interview_type, scheduled_at, interviewer_ids[], meeting_link}	POST /interviews/schedule	Creates Interview + links Interviewers. Generates .ics via calendar_utils. SES email with .ics attachment. Notification created.
Schedule (slot mode)	Bearer + JSON: {schedule_mode:'slots', interview_date, interview_type, interviewer_ids[]}	POST /interviews/schedule	Creates Interview + InterviewSlot rows. SES 'Select Slot' email sent to candidate with portal link.
Candidate confirms slot	Bearer + JSON: {slot_id}	PUT /interviews/{id}/confirm-slot	Slot marked confirmed. Interview.scheduled_at set. SES 'Slot Confirmed' + .ics sent. Returns {message}.
Reschedule interview	Bearer + JSON: {scheduled_at, meeting_link?}	PUT /interviews/{id}/reschedule	Updates scheduled_at. SES rescheduled email sent. Returns {message}.
Cancel interview	Bearer + path: interview_id	PUT /interviews/{id}/cancel	Sets status='cancelled'. SES cancellation email sent. Returns {message}.
Load interviewer list	Bearer token	GET /interviewers/	Returns [{id, name, email}] of all saved interviewers.

Frontend Action	Data Sent to Backend	API Endpoint	Response / Side Effects
Create interviewer	Bearer + JSON: {name, email}	POST /interviewers/	Creates Interviewer row (email unique). Returns {id, name, email}.
Delete interviewer	Path: interviewer_id	DELETE /interviewers/{id}	Deletes Interviewer row. Returns {message}.

15.9 Application Form Builder

Frontend Action	Data Sent to Backend	API Endpoint	Response / Side Effects
Form builder page loads	Bearer token	GET /jobs/my	Returns recruiter's jobs for the dropdown selector.
Save application form	Bearer + JSON array of questions: [{question_text, field_type, options, is_required, order_index}]	POST /jobs/{job_id}/application-form	Deletes all existing questions for this job. Inserts new set in order_index order. Returns saved list.

15.10 Job Sharing

Frontend Action	Data Sent to Backend	API Endpoint	Response / Side Effects
Share job with colleague	Bearer + JSON: [recruiter_id1, recruiter_id2, ...]	POST /job-shares/{job_id}/share	Validates caller is job owner. Creates/upserts JobShare rows. Sends SES email to each target. Returns {message}.
Shared jobs page loads	Bearer token	GET /job-shares/shared-with-me	Returns [{job_id, title, location}] of all jobs shared with this recruiter.

15.11 Recruiter Analytics

Frontend Action	Data Sent to Backend	API Endpoint	Response / Side Effects
Analytics page loads — call 1	Bearer token	GET /admin/recruiter/applications-per-job	Returns [{job_id, job_title, applications: N}] for own + shared jobs.
Analytics page loads — call 2	Bearer token	GET /admin/application-status-summary	Returns {applied:N, shortlisted:N, interview:N, offered:N, rejected:N}.

Frontend Action	Data Sent to Backend	API Endpoint	Response / Side Effects
Analytics page loads — call 3	Bearer token	GET /admin/upcoming-interviews	Returns [{candidate_name, job_title, scheduled_at, meeting_link}].
Analytics page loads — call 4	Bearer token	GET /admin/candidates-needing-action	Returns top 10 [{candidate_name, job_title, status, days_pending}] ordered by days_pending DESC.
Assign application	Bearer + path: application_id + ?recruiter_id	PUT /admin/assign-application/{id}	Validates auth. Assigns application. Returns {message, assigned_to}.

15.12 Notifications

Frontend Action	Data Sent to Backend	API Endpoint	Response / Side Effects
Notifications page loads	Bearer token	GET /notifications/	Returns [{id, title, message, is_read, created_at}] ordered newest first.
Dismiss/read notification	Bearer + path: notification_id	PUT /notifications/{id}/read	Sets is_read=True. Returns {message}.

16 Email Notification System

All transactional emails are sent via AWS SES using the send_email() or send_email_with_attachment() functions in email_utils.py. All email content is generated from centralised template functions in email_templates.py, each returning a (subject, body) tuple.

16.1 Email Template Reference

Function	Subject Prefix	Body Contains	Triggered By
application_confirmation(job_title, company_name)	Application Submitted	Job title, company, portal link	POST /applications/apply
shortlisted(job_title)	Shortlisted for {job}	Congratulations, next steps	Status update → shortlisted
rejected(job_title)	Application Update	Thank you message	Status update → rejected
offer(job_title)	Offer Letter	Selection confirmation	Status update → offered
interview_scheduled(...)	Interview Scheduled	Type, date/time, link/location	POST /interviews/schedule (direct)
interview_slots_shared(...)	Select Interview Slot	Portal link to pick slot	POST /interviews/schedule (slots)
interview_slot_confirmed(...)	Interview Slot Confirmed	Full details + .ics	PUT /interviews/{id}/confirm-slot
interview_rescheduled(...)	Interview Rescheduled	Old vs new schedule	PUT /interviews/{id}/reschedule
resume_uploaded()	Resume Uploaded	Upload confirmation	POST /resume/upload
recruiter_assigned(job_title)	Application Update	Recruiter assignment notice	POST /admin/auto-assign
job_shared(job_title, shared_by)	Job Shared With You	Job title, sender name	POST /job-shares/{id}/share

16.2 Calendar (.ics) Generation

When an interview is scheduled (direct mode) or a slot is confirmed, a .ics file is generated by generate_interview_ics() in calendar_utils.py and sent as an email attachment using send_email_with_attachment() via SES raw email.

ICS Field	Value
SUMMARY	Interview title (job title)
DESCRIPTION	Interview details (type, link, location)
DTSTART	scheduled_at formatted as YYYYMMDDTHHMMSS
DTEND	scheduled_at + 60 minutes (default duration)
PRODID	-//Recruitment Portal//Interview Calendar//EN

16.3 In-App Notification Creation

In-app notifications are created using create_notification() in notification_utils.py. This is called alongside every SES email trigger, creating a database record in the notifications table that the user sees when they visit the Notifications page.

```
# notification_utils.py
def create_notification(db: Session, user_id, title: str, message: str):
    notification = Notification(
        user_id=user_id,
        title=title,
        message=message,
    )
    db.add(notification)
    db.commit()
```

17 Analytics & Reporting

17.1 Candidate Profile Analytics

Endpoint: GET /candidate/profile-analytics | Cached: 120 seconds | Auth: Candidate only

Field	Source	Description
profile_views	COUNT(profile_view_events)	Number of times a recruiter loaded this candidate's profile
total_applications	COUNT(applications WHERE candidate_id=self)	Total number of jobs applied to
application_breakdown	GROUP BY application.status	Count per status: applied, shortlisted, interview, offered, rejected
profile_score	Calculated server-side (0–100)	40pts fields + 20pts skills + 20pts resume + 10pts experience + 10pts activity
upcoming_interviews	WHERE scheduled_at > NOW()	List of future interviews with job title, date, time

17.2 Recruiter Analytics Dashboard

Page: recruiter_analytics.py | 4 parallel API calls on load | Auth: Recruiter only

Widget	Data Source	What It Shows
Top metric cards	applications-per-job + status-summary	Total jobs, total applications, active status types
Plotly pipeline bar chart	application-status-summary	Candidate count at each stage: applied→shortlisted→interview→offered | rejected. Color-coded.
Key insights	status-summary aggregation	Total shortlisted, total interviews, total offers as metric cards
Applications per job list	recruiter/applications-per-job	Clickable job titles with app count. Click → navigates to that job's applicant view.
Upcoming interviews table	upcoming-interviews	All future interviews with candidate name, job, datetime, meeting link
Candidates needing action	candidates-needing-action	Top 10 candidates longest in active statuses with days_pending counter

END OF DOCUMENT

NMK Recruitment Portal — Complete Technical Documentation — v2.0